

Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

Généré le	2026-09-24T09:00:59.9848181Z
Commande	go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1
Plateforme	windows/amd64
Processeur	AMD Ryzen 7 7735HS with Radeon Graphics
Profil CPU	cpu-20260924-090100103.prof
Profil mémoire	memory-20260924-090100103.prof

1. Résultats du benchmark

Benchmark	Moyenne	ns/op	B/op	allocs/op
BenchmarkTick-16	6.64 ms	6635226	53018	17

Détail hyperfine

Métrique	Valeur
Moyenne	4.7468 s
Écart-type	n/a
Médiane	4.7468 s
Temps utilisateur	20.5541 s
Temps système	1.6187 s
Min	4.7468 s
Max	4.7468 s

Résumé CPU et mémoire

Mesure	Valeur
CPU échantillonné (pprof)	27.81s
Mémoire allouée (pprof)	99.86MB
Mémoire par opération	53018 B/op

Garbage collector Go

Cycles GC	64
Temps GC cumulé	66.040 ms

Plus longue pause	4.000 ms
Temps CPU GC cumulé	284.360 ms
Objectif mémoire maximal	95.00 MB

2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **27.81s**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).infectionCandidate	22.77s	81.88%	25.61s	92.09%
jeu-de-la-vie/game.(*Board).index (inline)	2.26s	8.13%	2.30s	8.27%
jeu-de-la-vie/game.(*Board).Step	1.22s	4.39%	1.91s	6.87%
runtime.asyncPreempt	0.35s	1.26%	0.35s	1.26%
math/rand.(*Rand).Float64 (inline)	0.33s	1.19%	0.64s	2.30%
jeu-de-la-vie/game.squaredDistance (inline)	0.22s	0.79%	0.22s	0.79%
math/rand.(*rngSource).Uint64 (inline)	0.19s	0.68%	0.19s	0.68%
jeu-de-la-vie/game.(*Board).countInfectionCand	0.17s	0.61%	25.78s	92.70%

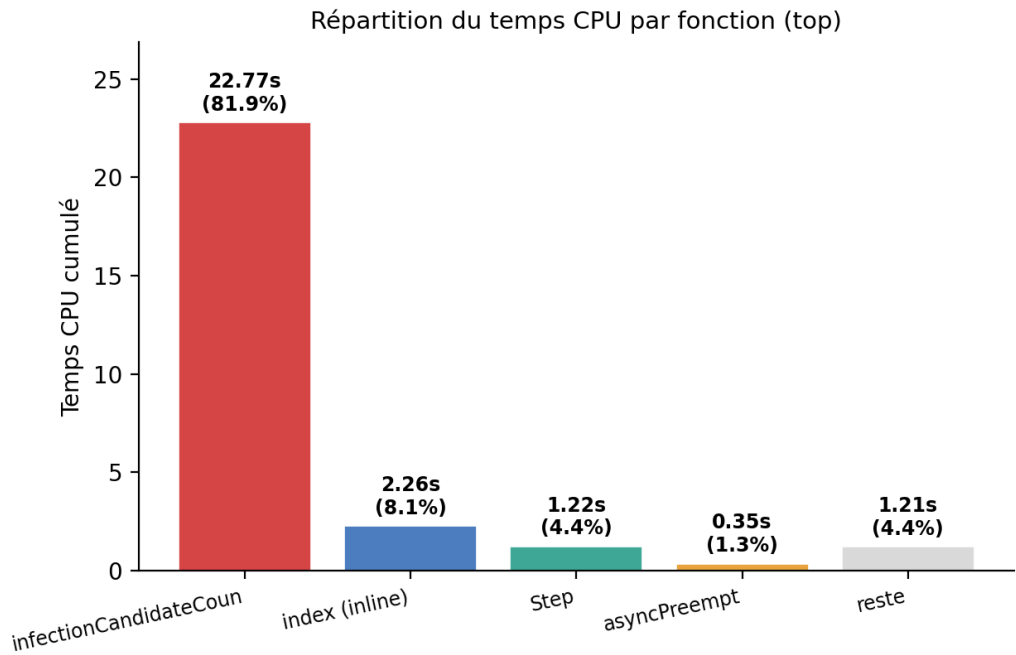


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

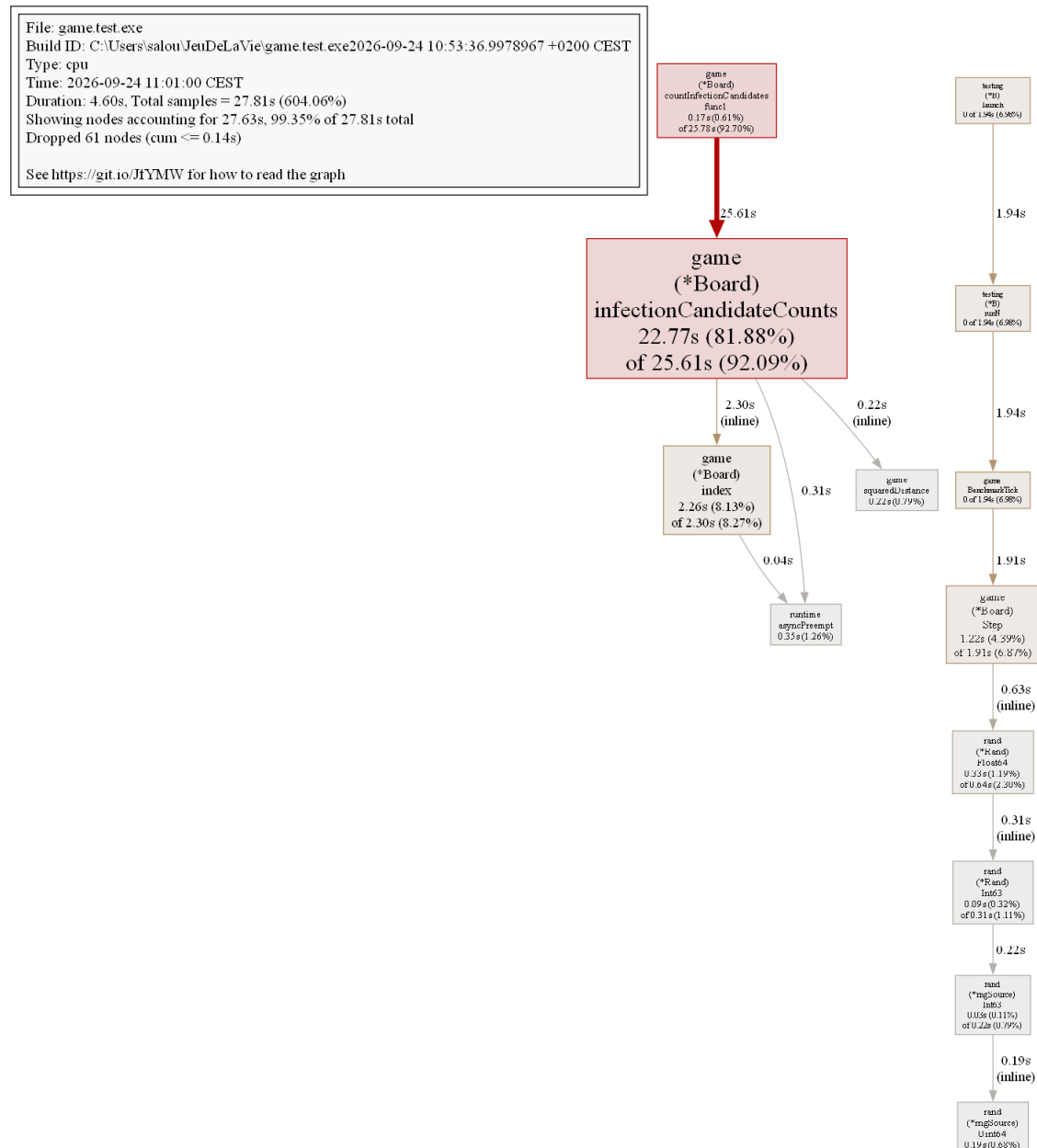


Figure — Graphe d'appels CPU (go tool pprof -png).

2.1 Diagnostic automatique

La fonction dominante de ce run est **jeu-de-la-vie/game.(*Board).infectionCandidateCounts**, responsable de **81.9%** du temps CPU total (flat), pour **92.1%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en $O(n)$ ou $O(\log n)$ équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

Rappel — pourquoi la complexité algorithmique domine
à mesure que les données grandissent (schéma, non mesuré)

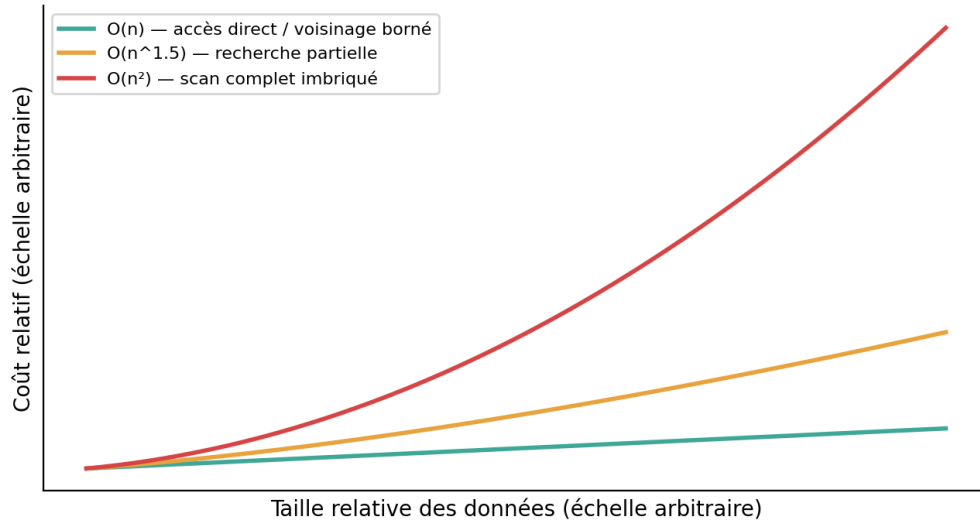


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **99.86MB**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).ensureScratch (inl	51.78MB	51.85%	51.78MB	51.85%
jeu-de-la-vie/game.NewBoard (inline)	19.33MB	19.36%	19.33MB	19.36%
jeu-de-la-vie/game.RandomBoard	16.57MB	16.59%	35.90MB	35.95%
runtime.mallocgc	3.51MB	3.51%	3.51MB	3.51%
jeu-de-la-vie/game.(*Board).countInfectionCand	2.50MB	2.50%	2.50MB	2.50%
runtime/pprof.StartCPUProfile	1.16MB	1.16%	1.16MB	1.16%
compress/flate.NewWriter (inline)	0.88MB	0.88%	1.52MB	1.52%
compress/flate.(*compressor).init	0.64MB	0.64%	0.64MB	0.64%

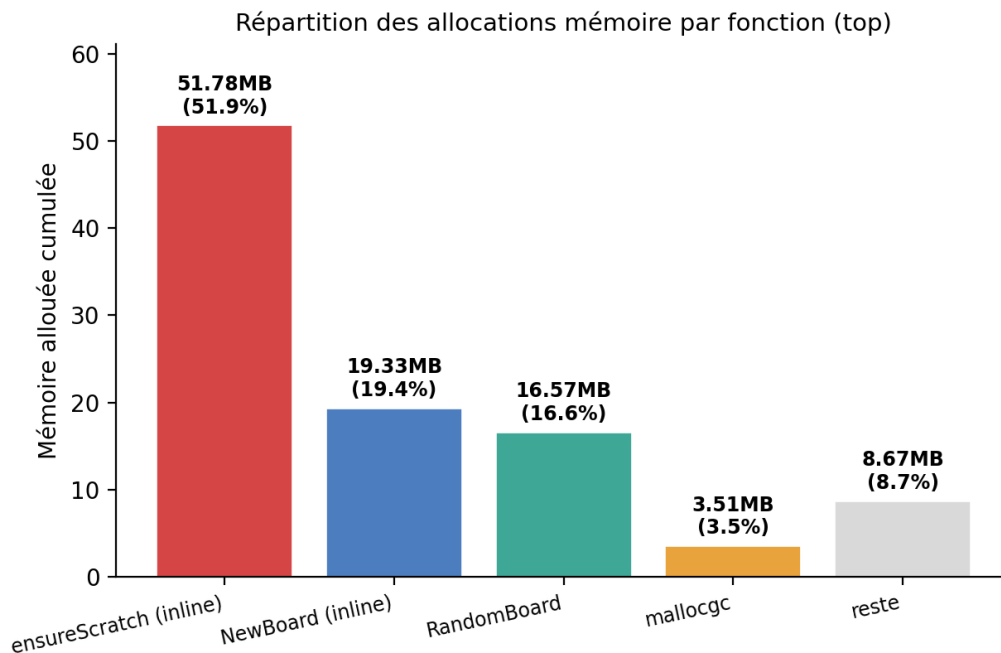


Figure — Répartition des allocations mémoire par fonction.

File: game.test.exe
 Build ID: C:\Users\salou\JeuDeLaVie\game.test.exe2026-09-24 10:53:36.9978967 +0200 CEST
 Type: alloc_space
 Time: 2026-09-24 11:01:05 CEST
 Showing nodes accounting for 91.18MB, 91.31% of 99.86MB total
 Showing top 9 nodes out of 80
 See <https://git.io/JfYMW> for how to read the graph

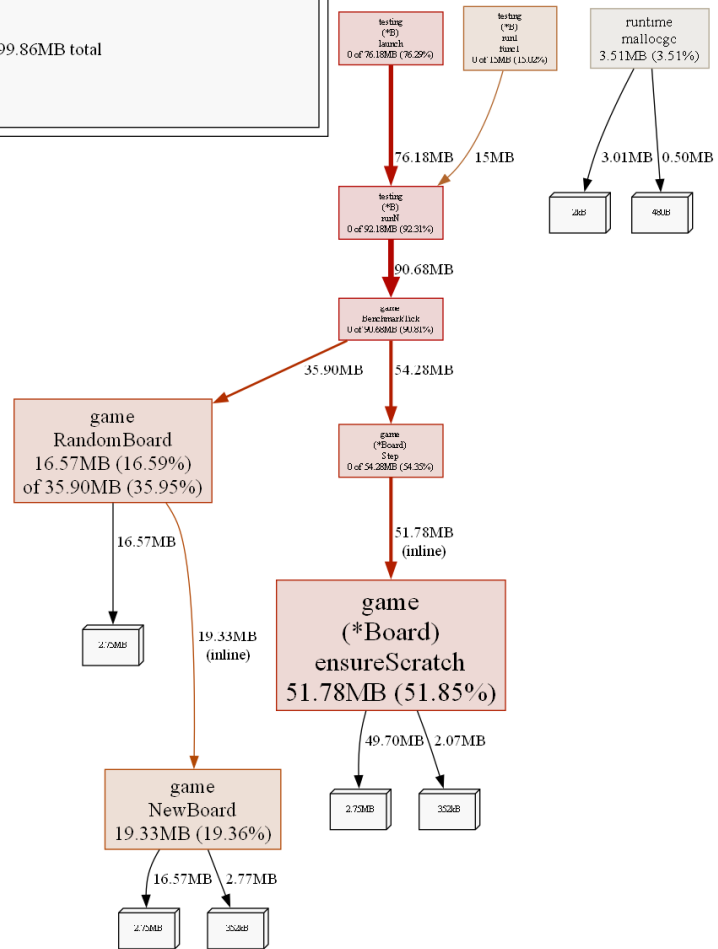


Figure — Graphe d'appels mémoire (go tool pprof -png).

4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Peu de runs (1).** Avec moins de 5 exécutions, aucune médiane/écart-type fiable n'est disponible. Recommandé : ≥ 10 runs (-count=10 ou hyperfine --min-runs 10).
- **Aucun warmup.** La première exécution peut être biaisée (cache froid, JIT du GC non stabilisé). Ajouter 1-2 warmups avant la mesure.
- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.

Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte 600×600 .
- La carte et le générateur aléatoire utilisent toujours la seed 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- La trace GC détaillée est enregistrée dans le fichier `gc-\*.log` et résumée dans le PDF.
- Les métriques I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

Rapport généré automatiquement le 2026-09-24 11:01:15 par generate_benchmark_report.py, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec go tool pprof.